PENETRATION TESTING REPORT:
OFFY_DIGITAL_VAULT (A BANKING SYSTEM).

Prepared By: Okunrobo Moses Emmanuel

Date: 1st August, 2026

1. EXECUTIVE SUMMARY

The security assessment of the Offy Digital Vault web application revealed several critical vulnerabilities. The most significant finding was a SQL Injection vulnerability on the user search endpoint, which, combined with Broken Object Level Authorization on the account and user-listing endpoints, allowed for complete extraction of the application's user database including every account's password, stored in plaintext. An Insecure Direct Object Reference (IDOR) flaw further allowed any authenticated user to view another user's balance and transaction data simply by altering an account identifier. The absence of rate limiting on the authentication endpoint also left the application exposed to automated credential brute-forcing. Collectively, these findings indicate the application lacks proper input sanitization, secure password storage, and consistent server-side authorization checks.

2. TARGET APPLICATION OVERVIEW

Offy Digital Vault is a full-stack banking web application built for structured penetration-testing practice. The frontend is implemented in React with Tailwind CSS and serves two portals a User Dashboard and an Admin Dashboard. The backend is a Java Spring Boot REST API responsible for authentication, account management, and transaction processing, backed by a MySQL/MariaDB database. The application was tested entirely within a local, isolated network consisting of a Kali Linux attacking VM and the Windows host running the application.

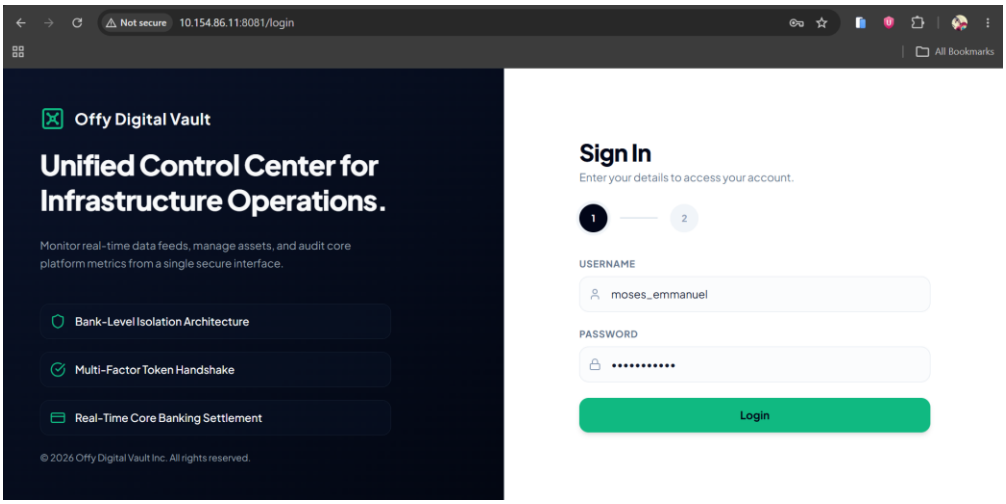

Figure 2.1 – User Login (Step 1: Credentials)

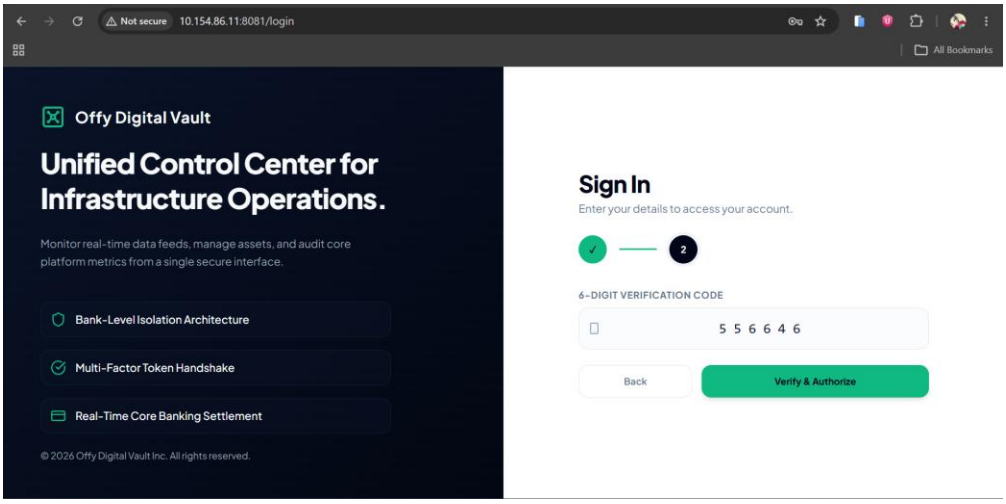

Figure 2.2 – User Login (Step 2: OTP Verification)

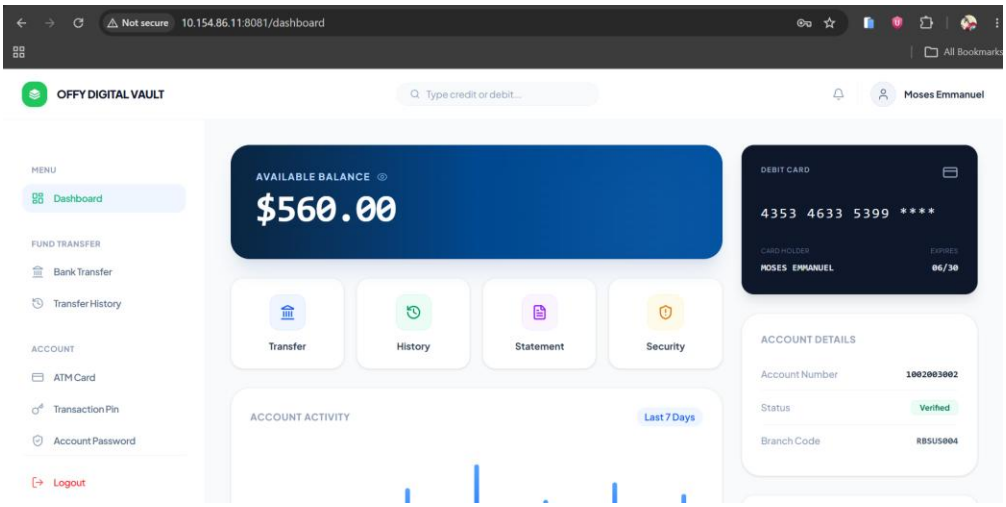

Figure 2.3 – User Dashboard (Balance & Account Overview)

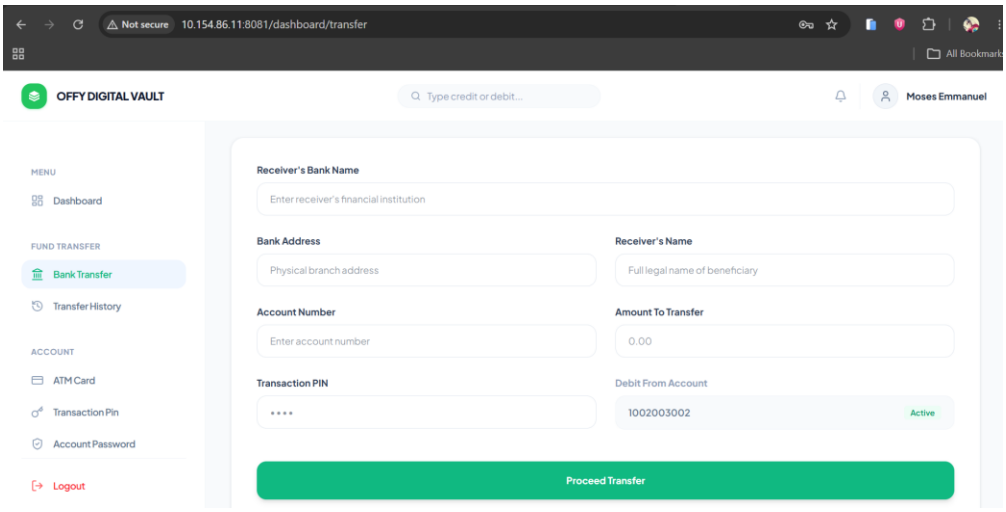

Figure 2.4 – Bank Transfer Form

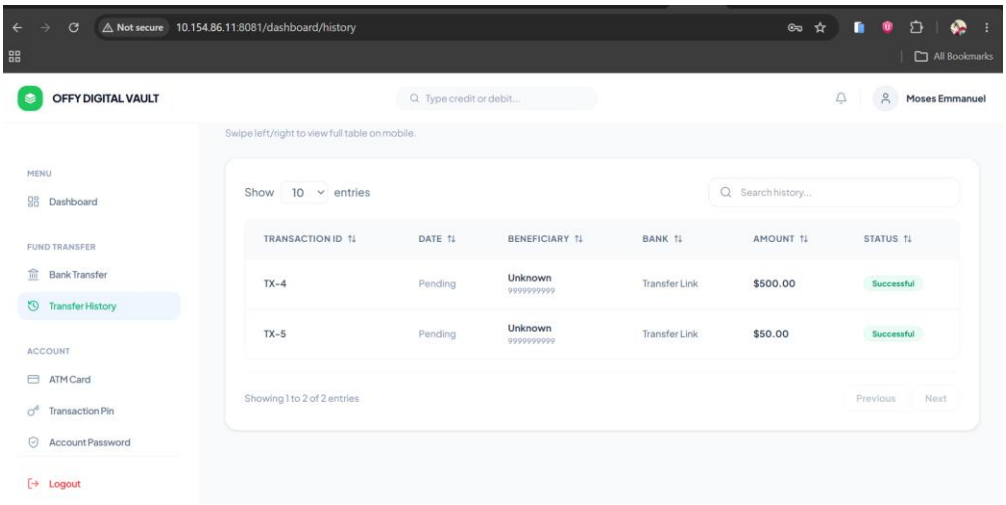

Figure 2.5 – Transfer History

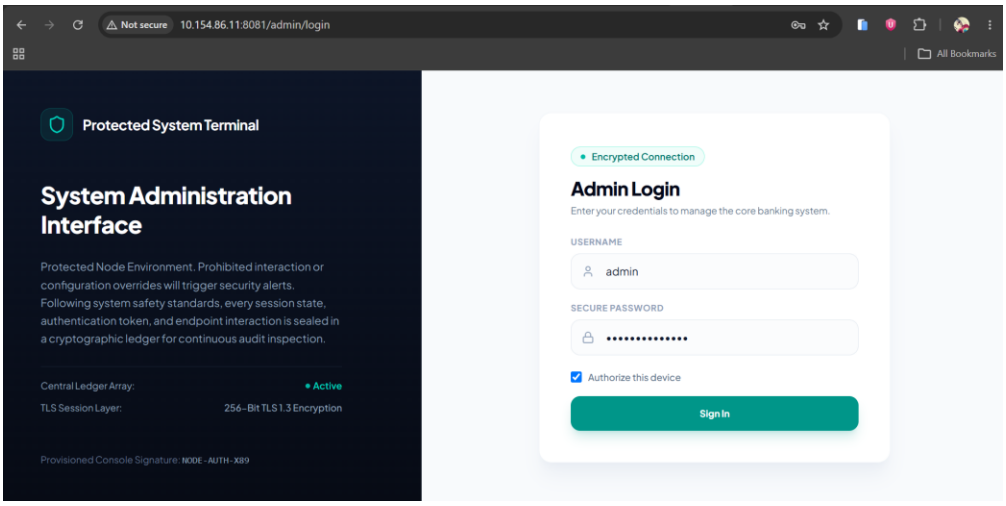

Figure 2.6 – Admin Login

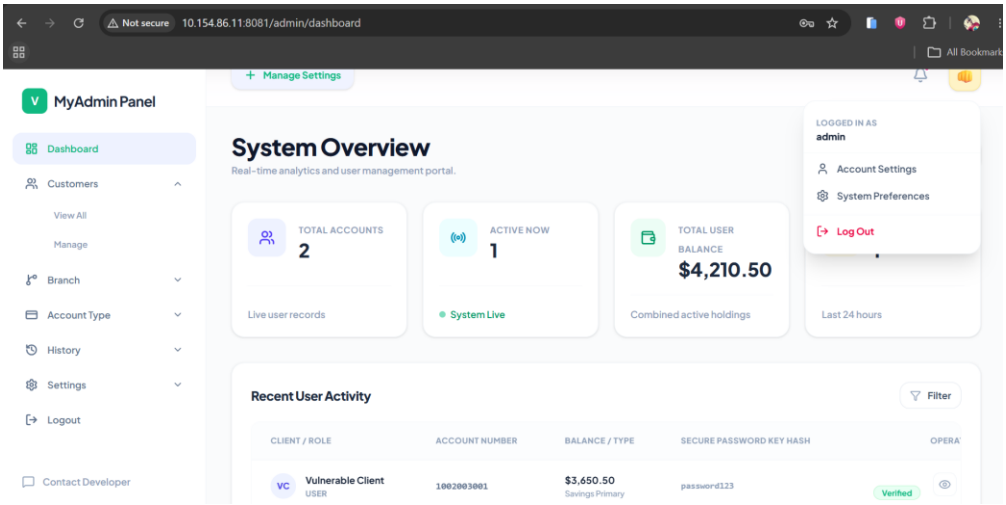

Figure 2.7 – Admin Dashboard (System Overview)

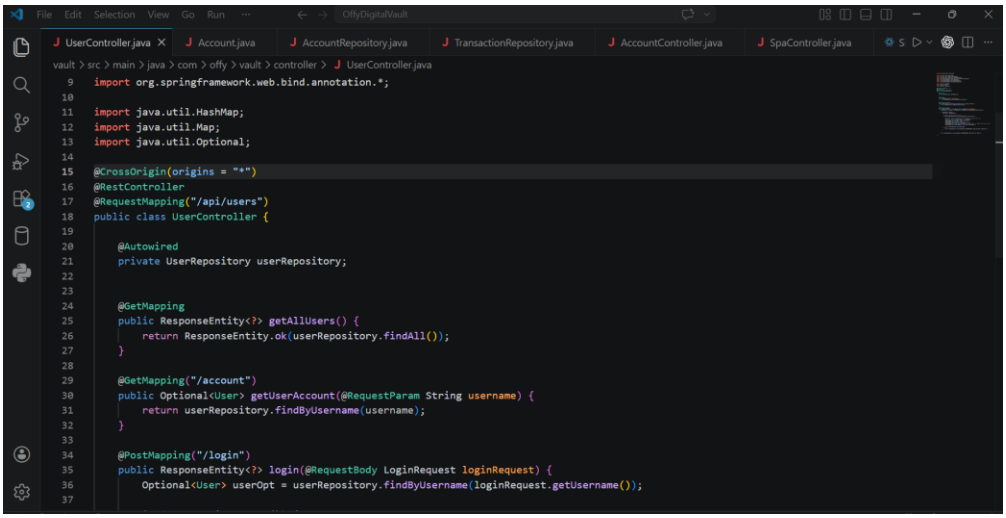

Figure 2.8 – Backend REST API (Java Spring Boot)

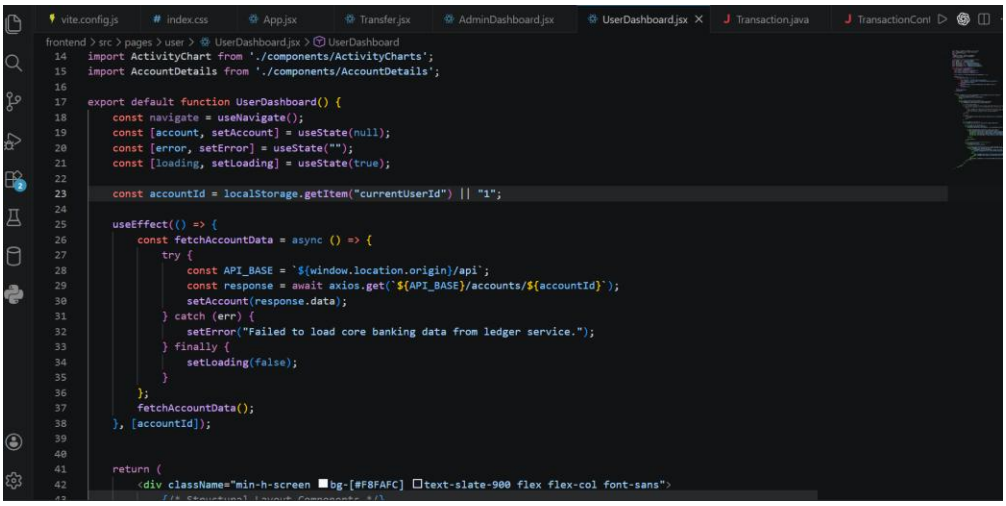

Figure 2.9 – Frontend Dashboard Component (React + Tailwind CSS)

3. VULNERABILITY SUMMARY TABLE

ID	Vulnerability	Severity	Status
001	Infrastructure & Service Enumeration (Nmap)	**Low**	Confirmed
002	Web Server Misconfiguration & Missing Security Headers (Nikto)	**Low**	Confirmed
003	API Endpoint Enumeration (Burp Suite)	**Informational**	Confirmed
004	Hidden Endpoint & Parameter Discovery (Python Fuzzing)	**Medium**	Confirmed
005	Insecure Direct Object Reference (IDOR) – Account Data	**High**	Confirmed
006	Broken Object Level Authorization – Mass User Data Exposure	**Critical**	Confirmed
007	Authentication Bypass via SQL Injection	**Critical**	Confirmed
008	Automated Database Exploitation & Schema Enumeration (SQLmap)	**Critical**	Confirmed
009	Sensitive Data Exposure – Plaintext Password Storage	**High**	Confirmed
010	Missing Brute-Force Protection on Authentication Endpoint	**High**	Confirmed

4. DETAILED FINDINGS

Finding 001: Infrastructure & Service Enumeration (Nmap)

Severity: Low

Description: A full TCP port sweep was performed against the application host to identify all exposed services prior to testing.

Methodology: A service- and version-detection scan was executed using Nmap:

```
nmap -sV -sC -p- 10.154.86.11 -T4
```

- Port 8081 – Apache Tomcat, hosting the React frontend and Spring Boot API under test.
- Port 3306 – MariaDB, reachable directly over the network and not restricted to localhost.
- Ports 80 / 443 – Apache/XAMPP hosting a legacy PHP module, with an expired, self-signed TLS certificate on 443.
- Port 8080 – A secondary Apache Tomcat instance.

Impact: Direct network exposure of the database service increases the risk of unauthorized connection attempts independent of the web application layer, and the expired self-signed certificate undermines transport confidentiality guarantees.

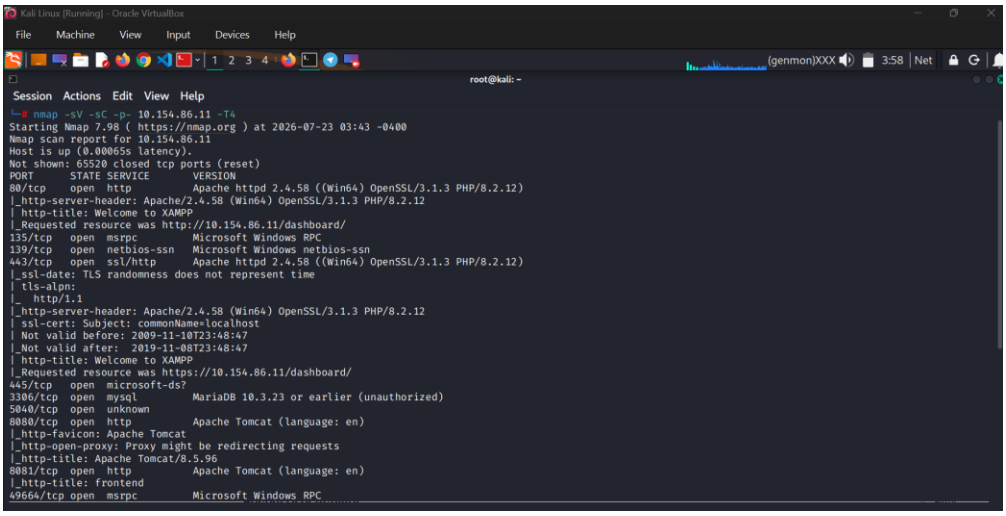

Figure 4.1 – Nmap Service and Version Scan

Finding 002: Web Server Misconfiguration & Missing Security Headers (Nikto)

Severity: Low

Description: An automated web server configuration scan was performed to identify missing hardening controls.

Methodology: The scan was executed using Nikto:

```
nikto -h http://10.154.86.11:8081/
```

- Missing security headers: Strict-Transport-Security, Content-Security-Policy, Permissions-Policy, and Referrer-Policy.
- X-Content-Type-Options header not set, permitting MIME-type sniffing by the browser.
- An uncommon /trace.axd endpoint disclosed to unauthenticated requests.

Impact: The missing headers reduce the browser's built-in protections against clickjacking, MIME confusion, and mixed-content attacks, broadening the impact of any client-side vulnerability.

Figure 4.2 – Nikto Web Server Scan

Finding 003: API Endpoint Enumeration (Burp Suite)

Severity: Informational

Description: The visible frontend routes (/login, /dashboard, /dashboard/transfer, /dashboard/history, /admin/login, /admin/dashboard) did not reveal the application's true REST API surface, so traffic was intercepted directly.

Methodology: Burp Suite's proxy was placed in front of the browser while manually exercising every user and admin flow. The resulting HTTP History and Site Map exposed the underlying API structure:

- GET /api/accounts/{id}, GET /api/accounts
- GET /api/transactions/transfer/history/{accountNumber}
- POST /api/users/login, GET /api/users

- GET /admin/login

Impact: Mapping the full API surface rather than relying on the frontend's visible navigation was the foundation for every exploitation step that followed.

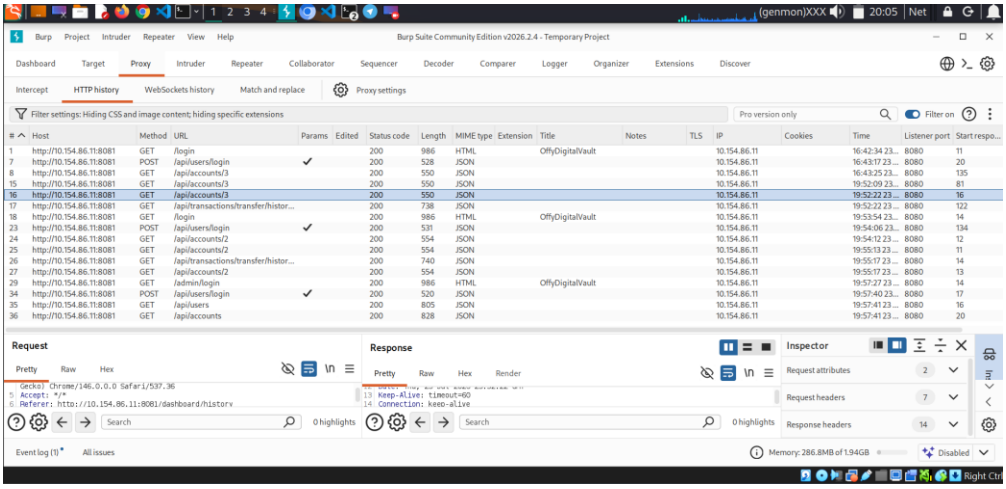

Figure 4.3 – Burp Suite HTTP History

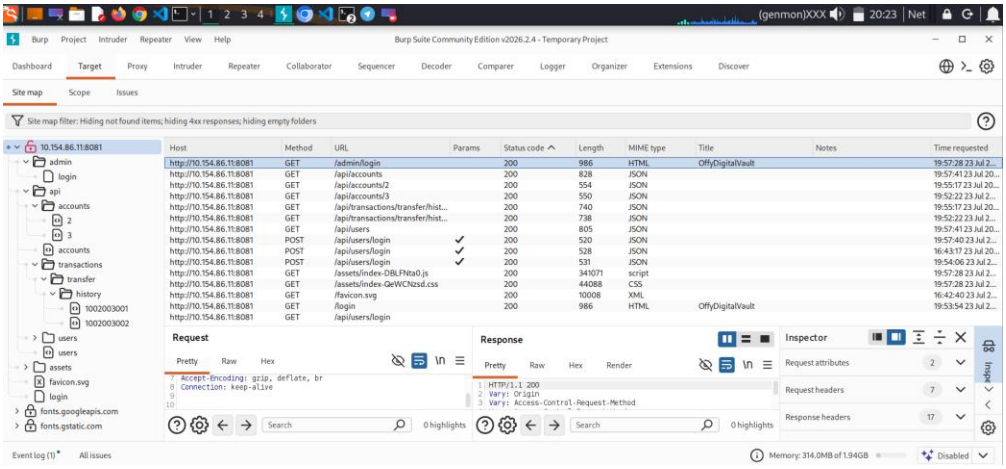

Figure 4.4 – Burp Suite Site Map

Finding 004: Hidden Endpoint & Parameter Discovery (Python Fuzzing)

Severity: Medium

Description: A custom Python script was used to fuzz the API for paths not linked anywhere in the frontend, using baseline response fingerprinting to filter out false positives.

```
import requests, hashlib

def check_path(path, baseline):
    base_status, base_len, base_hash = baseline
    url = f"{TARGET_URL}/{path}"
```

```
    r = requests.get(url, timeout=TIMEOUT, allow_redirects=False)
    body_hash = hashlib.md5(r.content).hexdigest()
    if r.status_code == base_status and body_hash == base_hash:
        return None
    if r.status_code in (200, 301, 302, 403):
        return (url, r.status_code, len(r.content))
```

Result: The fuzzer surfaced /api/accounts and /api/users independently of any UI link. Requesting /api/accounts directly returned a JSON array containing every account on the system, including the administrator's account balance and holder name.

Impact: The API exposes materially more data than the frontend deliberately shows, confirming that authorization is not being enforced consistently at the API layer.

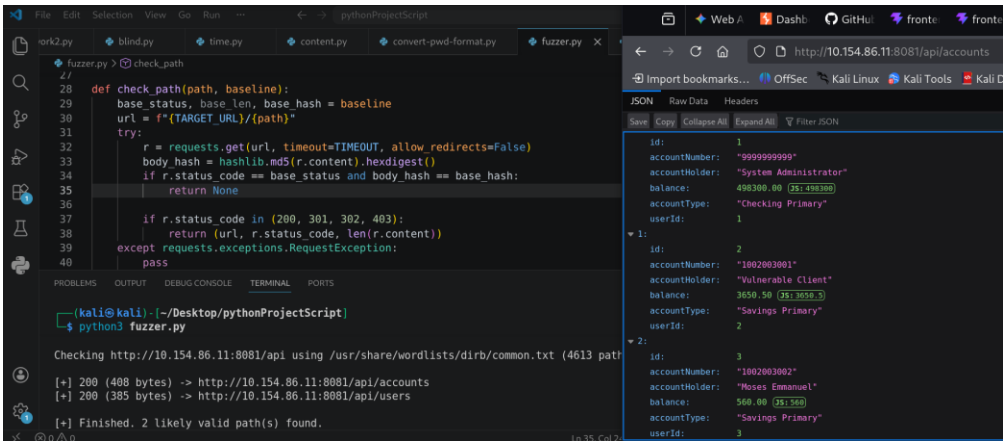

Figure 4.5 – Python Fuzzer Output and Leaked Account Data

Finding 005: Insecure Direct Object Reference (IDOR) – Account Data

Severity: High

Vulnerability Type: Broken Object Level Authorization (CWE-639)

Description: The /api/accounts/{id} endpoint does not verify that the account identifier being requested belongs to the session making the request.

Exploitation Methodology:

- Captured a legitimate request to the current user's own account: GET /api/accounts/3.
- Sent the request to Burp Suite Repeater.
- Changed the identifier to a different account: GET /api/accounts/2, without altering the session cookie.

- The server responded with 200 OK, returning the balance, account number, and holder name of a completely different user.

Impact: Any authenticated user can enumerate account identifiers sequentially to view every customer's balance and account details, violating data privacy and confidentiality controls expected of a financial application.

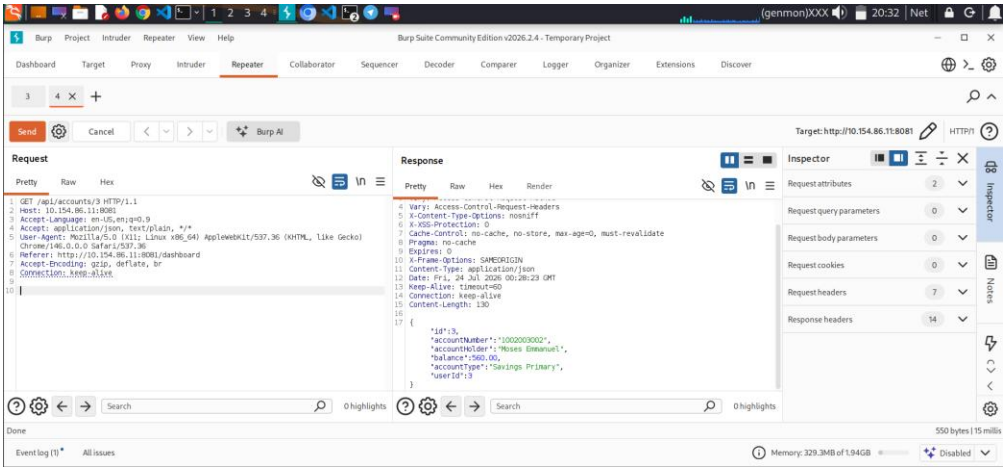

Figure 4.6 – Account ID Swapped in Burp Repeater

Finding 006: Broken Object Level Authorization – Mass User Data Exposure

Severity: Critical

Vulnerability Type: Broken Object Level Authorization / Excessive Data Exposure (OWASP API3:2023)

Description: The GET /api/users endpoint returns the complete user table to any authenticated caller, with no role check restricting it to administrators.

Exploitation Methodology: The endpoint was requested directly using a standard (non-admin) session token. The response returned all user records in full, including each user's id, username, plaintext passwordHash, accountNumber, balance, and role.

Impact: This finding, combined with Finding 007, represents a total compromise of the application's user base every account's login credentials and financial data are exposed in a single request.

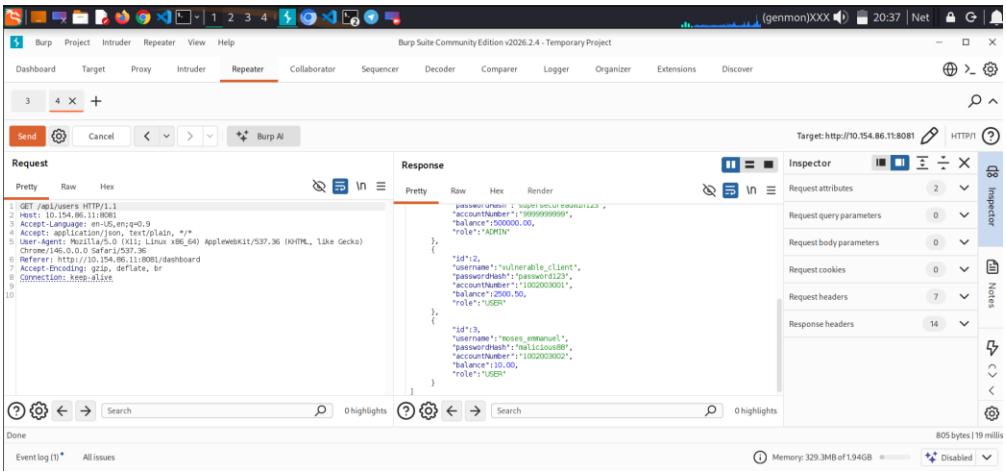

Figure 4.7 – /api/users Returns Every User Record

Remediation: Enforce role-based access control server-side on every endpoint that returns account or user data; never return password fields in any API response, hashed or otherwise.

Finding 007: Authentication Bypass via SQL Injection

Severity: Critical

Vulnerability Type: SQL Injection (CWE-89 / OWASP A03:2021-Injection)

Description: The GET /api/users/search endpoint builds its query by directly concatenating the username parameter into a native SQL string, with no parameterization or input sanitization.

Exploitation Methodology:

- A single quote appended to the username parameter produced a raw MariaDB syntax error in the response body, confirming the input reaches the query unsanitized.
- A boolean bypass payload (username=nonexistent' OR '1'='1) was submitted via a URL-encoded GET request, causing the WHERE clause to always evaluate true.
- The server returned every row in the users table in a single response, including plaintext passwords, for an endpoint that was intended to filter results by a single username.

Impact: The vulnerability allows complete, unauthenticated extraction of the user database without any specialized tooling a single crafted URL is sufficient.

Remediation: Replace string concatenation with parameterized queries (PreparedStatement) for every database-facing endpoint, and apply strict input validation on all user-controlled parameters.

Finding 008: Automated Database Exploitation & Schema Enumeration (SQLmap)

Severity: Critical

Vulnerability Type: SQL Injection – Automated Extraction (CWE-89)

Target Parameter: username, GET /api/users/search

Description: Following manual confirmation in Finding 007, the injection point was validated and automated using SQLmap to formally enumerate the backend database.

Methodology:

```
sqlmap -u "http://10.154.86.11:8081/api/users/search?username=test" --batch --dbs
sqlmap -u "..." --batch -D offy_digital_vault --tables
sqlmap -u "..." --batch -D offy_digital_vault -T users --dump
```

- SQLmap independently confirmed boolean-based blind, error-based, and time-based blind injection vectors on the username parameter.
- The backend database was identified as offy_digital_vault (MariaDB / MySQL).
- Automated dumping extracted the users, accounts, and support_tickets tables in full, matching the data obtained manually in Findings 006 and 007.

Impact: Automated confirmation validates that the vulnerability is reliably and repeatably exploitable, and demonstrates that an attacker requires no manual query-crafting skill to fully compromise the database.

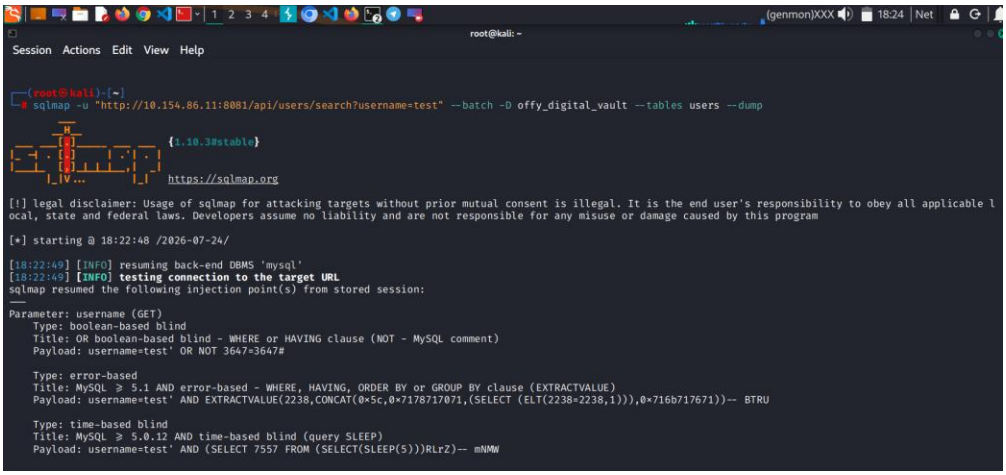

Figure 4.8 – SQLmap Confirms the Injection Point and Backend DBMS

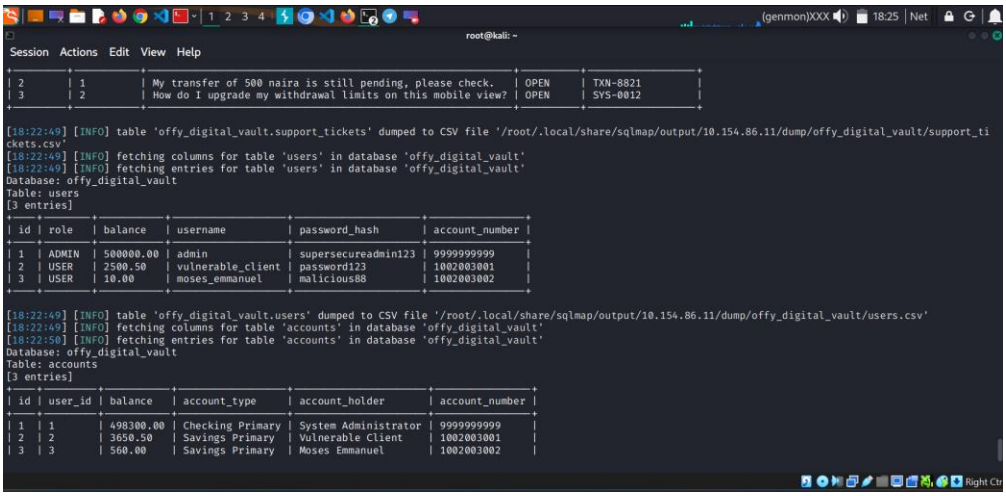

Figure 4.9 – Full users and accounts Tables Extracted

Finding 009: Sensitive Data Exposure – Plaintext Password Storage

Severity: High

Vulnerability Type: Cryptographic Failure (CWE-256 / OWASP A02:2021)

Description: The users table stores the passwordHash column in plaintext. No hashing algorithm, salting, or key-derivation function is applied before the password is persisted.

- admin → supersecureadmin123
- vulnerable_client → password123
- moses_emmanuel → malicious88

Impact: Because Findings 006 and 008 already demonstrate that this table can be fully extracted, plaintext storage removes any delay an attacker would otherwise face cracking hashed credentials every password is compromised the instant the data is read.

Remediation: Store passwords exclusively as salted hashes using a slow, purpose-built algorithm such as bcrypt or Argon2id. Never store or transmit the original password after registration.

Finding 010: Missing Brute-Force Protection on Authentication Endpoint

Severity: High

Vulnerability Type: Improper Restriction of Excessive Authentication Attempts (CWE-307)

Description: The POST /api/users/login endpoint accepts an unlimited number of authentication attempts from a single source with no rate limiting, delay, CAPTCHA, or account lockout.

Methodology: Hydra's http-post-form module was initially attempted but proved unreliable against this endpoint's JSON request body and 401 response handling. A JSON-native Python script was used instead:

```
r = requests.post(url, json={"username": username, "password": pwd}, timeout=5)
if r.status_code == 401:
    print(f"[-] failed: {pwd}")
else:
    print(f"[+] SUCCESS -> password: {pwd}")
```

Result: A short wordlist attack against a known test account recovered a valid password (password123 for vulnerable_client) directly, with no lockout or slowdown triggered at any point during the attack.

Impact: Combined with the plaintext-password disclosure in Finding 009, this confirms the login endpoint can be automated for account takeover at scale.

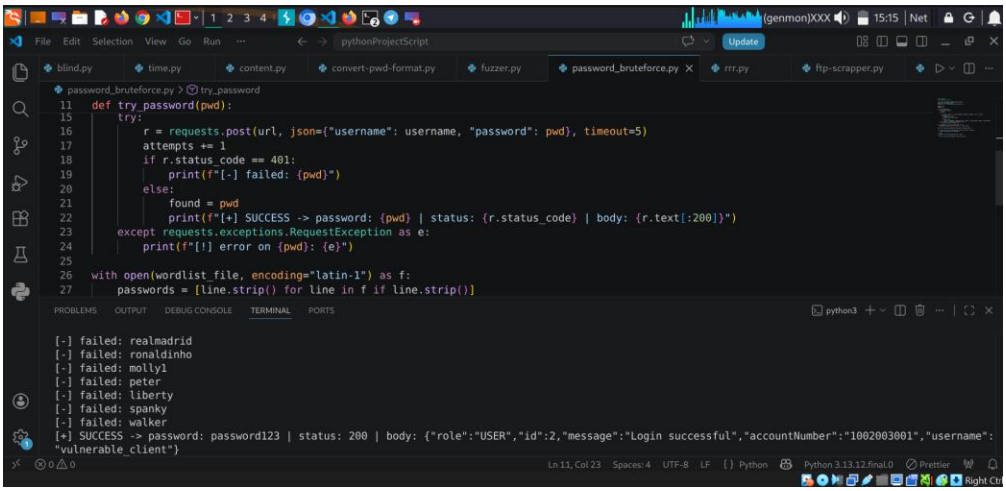

Figure 4.10 – Python Credential Brute-Force – Valid Password Recovered

Remediation: Implement rate limiting and progressive delays per account/IP, enforce account lockout after a defined threshold of failed attempts, and return a generic error message that does not confirm which field was incorrect.

5. CONCLUSION

The Offy Digital Vault application, in its current state, does not enforce the input validation, authorization, or credential-handling controls expected of a financial platform. The combination of SQL Injection, Broken Object Level Authorization, and plaintext password storage constitutes a critical, chained risk capable of exposing the entire user base with minimal attacker effort. It is recommended that all findings in this report particularly Findings 005 through 010 be remediated before any further development proceeds on top of the current authentication and data-access layers.